



FEU INSTITUTE OF TECHNOLOGY

GROUP 6

CCS0023L



TS21

Doc. Hadji Tejuco

John Joshua Esguerra	Jan Israel Seguiran	Patrick Simon Minimo	Dawn Andrei Pamesa
-------------------------	------------------------	-------------------------	-----------------------



Employee Attendance Tracker

Documentation:

The Attendance Tracker is a console-based Java application designed for managing employee data and attendance records. Its architecture is built upon four interconnected classes: `ATTENDANCERECORD`, `EMPLOYEE`, `HR`, and the execution class, `AttendanceTracker`. The system prioritizes data persistence, automatically loading and saving all operational data—employee details and attendance history—to a local text file named `employees.txt` upon startup and shutdown, respectively, ensuring data is retained across sessions.

The `ATTENDANCERECORD` class serves as the fundamental data unit, encapsulating the attendance status (a Boolean value) for a specific date. It includes methods for status retrieval and updating, along with static utility methods (`FROMSTRING` and `toString`) essential for serialization. The `EMPLOYEE` class aggregates an individual's data, including a unique `EMPLOYEEID`, name, and daily salary rate, and maintains a comprehensive internal list of their attendance records. This class houses the crucial business logic for attendance manipulation (marking, updating, and resetting records) and financial calculations, specifically determining monthly salary based on the count of present days within a specified month and year.

The `HR` class functions as the central administrative layer, responsible for managing the entire employee collection. It utilizes a `HashMap` (`EMPLOYEEMAP`) for efficient lookup of employees by ID and a `List` for iterative operations. Its responsibilities encompass employee lifecycle management (generating IDs, adding, and removing employees) and delegating attendance and salary requests to the appropriate `EMPLOYEE` instance.

The HR class is also the gatekeeper for file I/O, implementing the logic for structuring, saving, and safely parsing data from the persistence file, ensuring that the system state is accurately maintained.

The AttendanceTracker class contains the main execution method and governs the user interface flow, presenting a multi-level menu structure for accessing employee management, attendance tracking, salary calculation, and report generation functions. It employs a set of input validation utilities (READ_INT(), READ_MONTH(), etc.) to robustly handle user input, ensuring that data entered meets required format constraints and thereby preventing runtime errors. The system ensures a controlled exit by invoking the data saving mechanism before termination.

Test cases:

I. Employee Management (Add & Lookup)

- **Test: New Employee Addition**
 - **Action:** Add a new employee with a unique ID, Name, and Salary/Day (e.g., \$100).

```
--- EMPLOYEE ATTENDANCE TRACKER ---
1. EMPLOYEES
2. ATTENDANCE
3. CALCULATE MONTHLY SALARY
4. GENERATE MONTHLY ATTENDANCE REPORT
5. EXIT
CHOOSE AN OPTION: 1

--- EMPLOYEES MENU ---
1. LIST EMPLOYEES
2. ADD EMPLOYEE
3. REMOVE EMPLOYEE
4. BACK
CHOOSE AN OPTION: 2
ENTER NAME: Patrick Simon
ENTER SALARY PER DAY: 100
Employee added successfully with ID: 1
```

- **Test: Duplicate ID Rejection**
 - **Action:** Attempt to add a second employee using an ID that already exists.
 - N/A (ID is auto-assigned)
- **Test: Lookup Non-Existent ID**
 - **Action:** Attempt to perform an operation (like marking attendance or calculating salary) using an ID that has not been added to the system.

```
--- EMPLOYEE ATTENDANCE TRACKER ---
1. EMPLOYEES
2. ATTENDANCE
3. CALCULATE MONTHLY SALARY
4. GENERATE MONTHLY ATTENDANCE REPORT
5. EXIT
CHOOSE AN OPTION: 3
ENTER EMPLOYEE ID: 2
ENTER MONTH (1-12): 12
Employee not found!
```

II. Attendance Marking

- **Test: Basic Attendance Mark (Present)**

Action: Mark a valid employee as **Present** for a new date (e.g., "2025-11-01").

```
--- EMPLOYEE ATTENDANCE TRACKER ---
1. EMPLOYEES
2. ATTENDANCE
3. CALCULATE MONTHLY SALARY
4. GENERATE MONTHLY ATTENDANCE REPORT
5. EXIT
CHOOSE AN OPTION: 2

--- ATTENDANCE MENU ---
1. MARK ATTENDANCE
2. UPDATE ATTENDANCE
3. RESET ATTENDANCE
4. BACK
CHOOSE AN OPTION: 1
ENTER EMPLOYEE ID: 1
ENTER DATE (MM-DD): 11-01
ENTER STATUS (Present/Absent): Present
Attendance marked successfully for 2025-11-01
```

- **Test: Attendance Absent**

Action: Mark the same employee as **Absent** for a different, new date (e.g., "2025-11-02").

```
--- ATTENDANCE MENU ---
1. MARK ATTENDANCE
2. UPDATE ATTENDANCE
3. RESET ATTENDANCE
4. BACK
CHOOSE AN OPTION: 1
ENTER EMPLOYEE ID: 1
ENTER DATE (MM-DD): 11-02
ENTER STATUS (Present/Absent): Absent
Attendance marked successfully for 2025-11-02
```

- **Test: Duplicate Date Rejection**

Action: Attempt to mark attendance for the same employee on a date that already has a record (e.g., "2025-11-01").

```
--- ATTENDANCE MENU ---
1. MARK ATTENDANCE
2. UPDATE ATTENDANCE
3. RESET ATTENDANCE
4. BACK
CHOOSE AN OPTION: 1
ENTER EMPLOYEE ID: 1
ENTER DATE (MM-DD): 11-01
ENTER STATUS (Present/Absent): Present
Attendance for this date already marked!
```

III. Salary Calculation and Reporting

- **Test: Calculate Salary (Mixed Attendance)**

Pre-condition: Employee has 5 'Present' and 2 'Absent' records.
Salary/Day is \$100.

Action: Run the "Calculate monthly salary" option.

```
--- EMPLOYEE ATTENDANCE TRACKER ---
1. EMPLOYEES
2. ATTENDANCE
3. CALCULATE MONTHLY SALARY
4. GENERATE MONTHLY ATTENDANCE REPORT
5. EXIT
CHOOSE AN OPTION: 3
ENTER EMPLOYEE ID: 1
ENTER MONTH (1-12): 11
Patrick Simon's salary for 2025-11: 500.0
```

- **Test: Zero Attendance Salary**

Pre-condition: Employee has been added but has **no** attendance records.

Action: Run the "Calculate monthly salary" option.

```
--- EMPLOYEES MENU ---
1. LIST EMPLOYEES
2. ADD EMPLOYEE
3. REMOVE EMPLOYEE
4. BACK
CHOOSE AN OPTION: 2
ENTER NAME: Israel Seguiran
ENTER SALARY PER DAY: 300
Employee added successfully with ID: 3

--- EMPLOYEES MENU ---
1. LIST EMPLOYEES
2. ADD EMPLOYEE
3. REMOVE EMPLOYEE
4. BACK
CHOOSE AN OPTION: 4

--- EMPLOYEE ATTENDANCE TRACKER ---
1. EMPLOYEES
2. ATTENDANCE
3. CALCULATE MONTHLY SALARY
4. GENERATE MONTHLY ATTENDANCE REPORT
5. EXIT
CHOOSE AN OPTION: 3
ENTER EMPLOYEE ID: 3
ENTER MONTH (1-12): 11
Israel Seguiran's salary for 2025-11: 0.0
```

- **Test: Generate Comprehensive Report**

Pre-condition: Employee has 10 records total: 8 Present and 2 Absent.

Action: Run the "Generate attendance report" option.

```
--- EMPLOYEE ATTENDANCE TRACKER ---
1. EMPLOYEES
2. ATTENDANCE
3. CALCULATE MONTHLY SALARY
4. GENERATE MONTHLY ATTENDANCE REPORT
5. EXIT
CHOOSE AN OPTION: 4
ENTER MONTH (1-12): 11
Attendance Report for 2025-11
ID: 1, Name: Patrick Simon, Present: 8, Absent: 2
ID: 2, Name: Israel Seguiran, Present: 0, Absent: 0
```

IV. File Handling and Persistence

- **Test: Save Data on Exit**

Action: Add a new employee and mark several days of attendance. Then, select the **Exit** option (Option 5).

```
--- EMPLOYEES MENU ---
1. LIST EMPLOYEES
2. ADD EMPLOYEE
3. REMOVE EMPLOYEE
4. BACK
CHOOSE AN OPTION: 2
ENTER NAME: Joshua Esguerra
ENTER SALARY PER DAY: 200
Employee added successfully with ID: 3

--- EMPLOYEE ATTENDANCE TRACKER ---
1. EMPLOYEES
2. ATTENDANCE
3. CALCULATE MONTHLY SALARY
4. GENERATE MONTHLY ATTENDANCE REPORT
5. EXIT
CHOOSE AN OPTION: 5
Data saved. Exiting...
BUILD SUCCESSFUL (total time: 2 minutes 42 seconds)
```

- **Test: Load Data on Startup**

Pre-condition: Data has been successfully saved to employees.txt (as per F-01).

Action: Restart the program. Then, calculate the salary for the saved employee.

```

run:

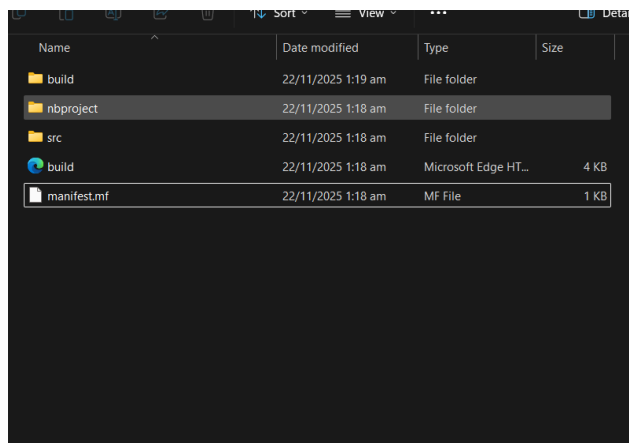
--- EMPLOYEE ATTENDANCE TRACKER ---
1. EMPLOYEES
2. ATTENDANCE
3. CALCULATE MONTHLY SALARY
4. GENERATE MONTHLY ATTENDANCE REPORT
5. EXIT
CHOOSE AN OPTION: 3
ENTER EMPLOYEE ID: 3
ENTER MONTH (1-12): 11
Joshua Esguerra's salary for 2025-11: 600.0

```

- **Test: First Run (No File)**

Pre-condition: Ensure employees.txt does not exist (delete it or run for the first time).

Action: Start the program.



```

run:

--- EMPLOYEE ATTENDANCE TRACKER ---
1. EMPLOYEES
2. ATTENDANCE
3. CALCULATE MONTHLY SALARY
4. GENERATE MONTHLY ATTENDANCE REPORT
5. EXIT
CHOOSE AN OPTION: |

```

Task Scheduler for a Project Management Tool

Documentation:

The Task Management System is a console-based Java application designed for overseeing project tasks, team assignments, and productivity tracking. Its architecture revolves around five interconnected classes: `LogInSystem`, `Task`, `TeamMember`, `ProjectManager`, and the execution class, `MainProgram`. The system emphasizes role-based access (project managers and team members), data persistence, and operational integrity, automatically loading and saving all data—login credentials, tasks, and team member details—to local text files (e.g., `tasks.txt` and `teamMembers.txt`) upon startup and shutdown, ensuring continuity across sessions.

The `LogInSystem` class handles user authentication, encapsulating login credentials including username, password, and user type (Project Manager or Team Member). It provides methods for verifying login attempts and supports serialization for data persistence, acting as the entry point for secure access to the system.

The `Task` class represents the core data unit for individual tasks, encapsulating attributes such as a unique `taskId`, description, `assignedTo` (team member ID), status (e.g., Not Started, In Progress, Completed), and priority (Low, Medium, High). It includes getters, setters with validation (e.g., for status and priority), and a `toString` method for serialization, enabling efficient storage and retrieval of task details.

The `TeamMember` class aggregates data for individual team members, including a unique `memberID`, name, and an internal list of assigned tasks. It enforces capacity constraints (limiting members to 5 in-progress tasks) through methods like `assignTask` and `canAcceptTask`, while providing utilities for status counting (e.g., completed and pending tasks) and productivity reporting. The class also supports serialization via `toString` for data persistence.

The ProjectManager class serves as the central administrative layer, managing collections of tasks and team members under a specific manager. It utilizes an ArrayList for tasks and team members, alongside a HashMap for efficient task lookups by ID. Responsibilities include task lifecycle management (adding, assigning, and updating tasks), team member oversight, report generation (e.g., productivity metrics), and file I/O for saving/loading data. It delegates operations to TeamMember and Task instances while ensuring constraints like task uniqueness and member capacity.

The MainProgram class contains the main execution method and orchestrates the user interface flow, presenting role-specific menu structures for project managers (e.g., adding tasks, assigning to members, updating statuses) and team members (e.g., updating their assigned tasks). It employs input validation utilities (e.g., parsing integers and handling exceptions) to manage user input robustly, preventing errors. The system supports controlled exits by invoking data saving mechanisms, maintaining system state integrity across sessions.

Test cases:

I. Login Testing

- **Test: Valid Username - Wrong Password**

```
=== TASK MANAGEMENT SYSTEM ===  
Enter username: manager1  
Enter password: pass2  
Are you a (1) Project Manager or (2) Team Member? 1  
Invalid username or password!  
Thank you for using the Task Management System!  
BUILD SUCCESSFUL (total time: 17 seconds)
```

- **Test: Valid Credentials - Wrong Role**

```
=== TASK MANAGEMENT SYSTEM ===  
Enter username: manager1  
Enter password: mgrpass1  
Are you a (1) Project Manager or (2) Team Member? 2  
Invalid username or password!  
Thank you for using the Task Management System!  
BUILD SUCCESSFUL (total time: 11 seconds)  
|
```

- **Test: Successful Login**

```
=== TASK MANAGEMENT SYSTEM ===  
Enter username: manager1  
Enter password: mgrpass1  
Are you a (1) Project Manager or (2) Team Member? 1  
Login successful! Welcome Project Manager
```

II. Add Task

- **Test: Invalid Task ID**

```
=== PROJECT MANAGER MENU ===  
1. Add a new task  
2. Assign a task to a team member  
3. Update Task Status  
4. View all tasks  
5. Generate Productivity Report  
6. Exit  
Choose an option: 1  
Enter task ID: invalid  
Invalid task ID format!
```

- **Test: Duplicate Task ID**

```

=== PROJECT MANAGER MENU ===
1. Add a new task
2. Assign a task to a team member
3. Update Task Status
4. View all tasks
5. Generate Productivity Report
6. Exit
Choose an option: 1
Enter task ID: 1
Enter task description: test
Enter priority (Low/Medium/High): High
Task added successfully!

=== PROJECT MANAGER MENU ===
1. Add a new task
2. Assign a task to a team member
3. Update Task Status
4. View all tasks
5. Generate Productivity Report
6. Exit
Choose an option: 1
Enter task ID: 1
Enter task description: Test
Enter priority (Low/Medium/High): High
Task ID already exists!

```

III. Assign Task

- Test: Assign Task to Team Member

```

=== ALL TASKS ===

```

Task ID	Description	Assigned To	Status	Priority
67	Finals For OOP	TM002	Completed	High
21	Finish CSA Finals	TM005	Completed	Medium
1	test	Unassigned	Not Started	High

```

Enter task ID to assign: 1
Available team members:
TM001 - Alice Developer (Can accept task: Yes)
TM002 - Charlie Tester (Can accept task: Yes)
TM003 - David Analyst (Can accept task: Yes)
Enter member ID: TM001
Task assigned successfully to Alice Developer

```

- Test: Assign same task to different Team Member

```

=== ALL TASKS ===
Task ID      Description                Assigned To    Status        Priority
-----
67           Finals For OOP                  TM002         Completed     High
21           Finish CSA Finals              TM005         Completed     Medium
1            test                          TM001         Not Started   High
Enter task ID to assign: 1
Available team members:
TM001 - Alice Developer (Can accept task: Yes)
TM002 - Charlie Tester (Can accept task: Yes)
TM003 - David Analyst (Can accept task: Yes)
Enter member ID: TM002
Task assigned successfully to Charlie Tester

```

IV. Update Task Status

- Test: Update status of an Unassigned task

```

=== ALL TASKS ===
Task ID      Description                Assigned To    Status        Priority
-----
67           Finals For OOP                  TM002         Completed     High
21           Finish CSA Finals              TM005         Completed     Medium
1            test                          TM002         Completed     High
2            Test                          Unassigned     Not Started   Low
Enter task ID to update: 2
Enter new status (Not Started/In Progress/Completed): Completed
Task status updated successfully!

```

- Test: Update status to exact same status

```

=== ALL TASKS ===
Task ID      Description                Assigned To    Status        Priority
-----
67           Finals For OOP                  TM002         Completed     High
21           Finish CSA Finals              TM005         Completed     Medium
1            test                          TM001         Completed     High
2            Test                          Unassigned     Completed     Low
Enter task ID to update: 2
Enter new status (Not Started/In Progress/Completed): Completed
Task status updated successfully!

```

